● INDIAN INSTITUTE OF INFORMATION TECHNOLOGY RANCHI • CSE MINOR PROJECT

Adaptive Self-Consistency in Large Language Models

Mid-Term Presentation Script, Slide-by-Slide Defense Guide & Faculty Q&A;

Supervisor: Dr. Bharat Singh, Assistant Professor, Dept. of CSE | **Evaluation Date:** November 12, 2026
Student Team: Pankaj Jangid (2023UG3004) • Aagman (2023UG3027) • Abhinav Kumar (2023UG4018)
Presentation Deck: 16 Slides (presentation.pptx) | **Total Time:** 15–18 Minutes (~5–6 minutes per speaker)

Speaker	Student Name & Reg. No.	Presentation Part	Slides	Core Responsibility & Narrative
Speaker 1	**Pankaj Jangid** 2023UG3004	**Part 1: Motivation & Foundation**	**Slides 1 – 5** (~5 min)	Opening context, why greedy decoding fails (Janet's eggs), Wang et al. (ICLR 2023) foundation, the 3 flaws of static N=40 sampling, and research objectives.
Speaker 2	**Aagman** 2023UG3027	**Part 2: Methodology & Engineering**	**Slides 6 – 10** (~5.5 min)	5-Step adaptive evaluation loop, dual stopping gates (margin >= 2 + confidence >= 70%), math formulation, codebase modules, and 6-stage regex parser.
Speaker 3	**Abhinav Kumar** 2023UG4018	**Part 3: Empirical Results & Roadmap**	**Slides 11 – 16** (~5.5 min)	Replication & noise dilution (Fig 1), Pareto frontier & 48.4% token compute reduction (Fig 2), sample allocation histogram (Fig 3), ablations, milestones, roadmap & Q&A.;

PART 1: MOTIVATION & FOUNDATION | Speaker: Pankaj Jangid (Slides 1 – 5)

Slide 1: Title & Project Introduction

[Slide Cue]: *Title slide displayed with institutional badge, student names, and supervisor Dr. Bharat Singh.*

"Respected Supervisor Dr. Bharat Singh sir, honorable faculty members, and fellow students — good morning. I am Pankaj Jangid, and along with my team members Aagman and Abhinav Kumar, we are presenting our B.Tech CSE Minor Project titled: 'Adaptive Self-Consistency in Large Language Models: Dynamic Early Stopping for Efficient and Accurate Multi-Step Reasoning'. Today, LLMs are widely deployed across math problem solving, coding, and decision reasoning. However, inference costs and computational waste remain massive bottlenecks. In this mid-term evaluation, we demonstrate how our dynamic, confidence-guided early stopping mechanism cuts inference compute by 48.4% while preserving 100% of reasoning accuracy. Let us begin by examining why current LLMs make mistakes in math problems."

Key Emphasis: Introduce team with confidence; set clear expectations of the 48.4% compute reduction result upfront.

Slide 2: The Problem: Why LLMs Make Mistakes in Math Problems

[Slide Cue]: *Flaws of greedy decoding on left card; Janet's egg problem and 3 diverse reasoning paths on right card.*

"When we ask a modern language model to solve a mathematical word problem, we prompt it using Chain-of-Thought — meaning we tell the model to 'think step by step'. This works well, but standard greedy decoding has a fatal flaw: there are no second chances. In greedy decoding, the model always picks the single most probable token at each step. If the model makes a small arithmetic slip on step 2, every subsequent step is poisoned. Like a student writing a sign error on line two of an exam, the final numerical answer is ruined despite a good overall plan. Even worse, the model exhibits false confidence — it outputs an incorrect number with the exact same high probability as a correct one. Notice the real example on the right: Janet has 16 eggs, eats 3, bakes with 4, and sells the rest at $2 each. Path 1 subtracts sequentially and gets $18 (correct). Path 2 groups items first and gets $18 (correct). But Path 3 makes an arithmetic slip (16 - 7 = 8) and outputs $16 (flawed). Here is our core intuition: Correct reasoning paths arrive at the exact same consensus ($18). Incorrect attempts make random arithmetic mistakes ($16). This brings us to the foundation of our work: Self-Consistency."

Key Emphasis: Point to Path 1, 2, and 3. Emphasize that correct paths agree, while incorrect paths make random errors.

Slide 3: The Existing Foundation: Self-Consistency (Wang et al., ICLR 2023)

[Slide Cue]: *Majority voting and logprob weighting rules on left; Wang et al. published benchmark table on right.*

"To solve greedy decoding errors, Wang et al. introduced Self-Consistency at ICLR 2023. Instead of querying the model only once, they sample multiple diverse reasoning attempts at temperature T=0.7, extract the numerical answer from each path, and take a majority vote. They also formulated a confidence-weighted rule, where each path is weighted by the exponential of its average token log-probability, selecting the answer with highest accumulated probability mass. As you can see in the published table on the right, this simple idea delivered massive gains across every benchmark: PaLM-540B jumped from 56.5% to 74.4% on GSM8K — an absolute gain of +17.9%. SVAMP gained +7.6%, and AQuA gained +12.5%. GPT-3 similarly improved by +17.9%. Self-Consistency acts as an unsupervised verifier without requiring extra fine-tuning or human labeling. However, it introduces a severe practical drawback."

Key Emphasis: Highlight the +17.9% gain on GSM8K in the table. Emphasize that Self-Consistency is an unsupervised verifier.

Slide 4: The Practical Limitation: The Static Budget Tax

[Slide Cue]: *Three pillar cards: The One-Size-Fits-All Waste, Diminishing Returns (Plateau), and Noise in the Tail (Dilution).*

"In standard self-consistency, the number of paths is fixed in advance — typically N=10 to N=40 paths for every single problem unconditionally. We identified three critical flaws with this static approach: First, The One-Size-Fits-All Waste: If a question is elementary — for instance, 'What is 15 - 7?' — the first two generated paths already agree with 100% certainty. Forcing the system to generate 38 more paths is completely pointless. Up to 80 to 90% of token compute on easy problems is wasted confirming what was already obvious. Second, Diminishing Returns (The Plateau): In Wang et al.'s own scaling curves, the steepest accuracy jump occurs within the first 3 to 5 paths. Scaling all the way from 10 to 40 paths yields less than a 1.5% accuracy gain, while multiplying API cost and latency by 40 times. Third, Noise in the Tail (Dilution): When sampling at temperature 0.7, generating 40 paths eventually introduces hallucinations and spurious agreement. In our own benchmark, accuracy peaked at 4 paths, but degraded at 6 paths because noisy tails polluted the vote. This led directly to our research question."

Key Emphasis: Stress the term 'Static Budget Tax'. Clearly distinguish easy questions from hard questions.

Slide 5: Our Research Objectives & Handoff

[Slide Cue]: *Core research question banner at top; 3 target cards in middle; practical deliverables at bottom.*

"Our project answers one fundamental question: 'Can an LLM recognize when it has reached a clear consensus, allowing it to stop generating early and cut compute by over 40% without losing reasoning accuracy?' We established three concrete technical targets: 1. An Efficiency Target: Reduce completion tokens by at least 40% compared to full-budget sampling. 2. A Safety Target: Guarantee 0.0% accuracy regret — meaning our early exit must never sacrifice correctness on any problem that the full budget would have solved. 3. A Decision Criteria: Build a dual gate that checks both discrete vote lead and the model's token-level certainty. In practice, this means faster response times for users, cutting cloud API bills in half, and preventing rate-limit exhaustion. I will now hand over to Aagman, who will walk us through our methodology, mathematical stopping rules, and system architecture."

Key Emphasis: Seamless verbal handoff: 'I will now hand over to Aagman...'

PART 2: METHODOLOGY & ENGINEERING | Speaker: Aagman (Slides 6 – 10)

Slide 6: Proposed Methodology: Confidence-Aware Adaptive Self-Consistency

[Slide Cue]: *5-Step evaluation loop on left card; Dual stopping criteria (Lead Margin >= 2 AND Confidence Share >= 70%) on right card.*

"Thank you, Pankaj. Good morning respected committee members. To achieve dynamic early stopping, we designed Confidence-Aware Adaptive Self-Consistency (CA-ASC). Instead of querying all paths up front, our system operates in a sequential, micro-batch loop: Step 1: We sample a small batch of just 2 paths (b=2) at temperature 0.7. Step 2: We extract the numerical answers using our robust regex parser. Step 3: We compute the sequence confidence weight of each path using the model's token log-probabilities. Step 4: We evaluate our Dual Stopping Gates. Step 5: If both gates pass, we terminate immediately and output the consensus answer. If there is a tie or low confidence, we generate another pair of paths, repeating up to a maximum safety budget of 6. Now, look at the right card. Why do we need two stopping criteria instead of just one? Condition 1 (Lead Margin >= 2): The top candidate must lead the runner-up by at least 2 votes. If the first 2 paths agree (2 vs 0), the margin is 2, passing Gate 1. If they split (1 vs 1), the margin is 0, so the loop continues. Condition 2 (Confidence Share >= 70%): The leading answer's total accumulated token probability weight must account for at least 70% of all generated paths. This second gate is our safety net: even if two paths coincidentally produce the same number, if the model was hesitant or had low token probabilities, the system refuses to stop early."

Key Emphasis: Emphasize why both gates are required: discrete margin prevents ties, confidence share filters out lucky guesses.

Slide 7: Mathematical Formulation & Theoretical Guarantee

[Slide Cue]: *Mathematical definitions of Margin Delta, Sequence Weight w_i, and Dual Rule on left; Exponential risk bound on right.*

*"On Slide 7, we present the formal mathematical foundation of our stopping policy. First, for any candidate answer a, let c(a) be the number of paths that arrived at that answer. The agreement margin Delta_t after t generated paths is: Delta_t = c_(1) - c_(2), where c_(1) is the vote count of the leader, and c_(2) is the runner-up. Second, we compute each path's sequence weight w_i by length-normalizing the sum of generated token log-probabilities: w_i = exp((1 / |r_i|) * sum log P(y_k | y_= 2 AND (sum_{lead} w_i / sum_{all} w_j) >= 0.70. On the right, we show the theoretical risk bound. Under Hoeffding's inequality, the probability of an early stopping error decays exponentially: P(Early Stop Error) <= exp(- Delta_t^2 / 2t) * (1 - S_conf). As the margin grows to 2, the chance that the true answer is hidden in the unseen minority drops exponentially. Best of all, evaluating these rules takes under 0.05 milliseconds on a CPU — adding virtually zero overhead."*

Key Emphasis: Read the formulas clearly. Explain that length normalization prevents penalizing longer, rigorous proofs.

Slide 8: System Architecture & Engineering Modules

[Slide Cue]: *Three codebase module cards: client_wrapper.py (Async API Client), extractor.py (Robust Parser), and ca_asc_evaluator.py (Offline Replay Engine).*

"To turn this theoretical formulation into a working system, we built three core engineering modules: 1. client_wrapper.py: This is our asynchronous inference harness. Cloud APIs enforce strict rate limits (RPM and TPM). We built a concurrent worker pool with automatic rate-limit backoff: if an HTTP 429 response is received, it pauses for 15 seconds to let the token bucket refill. Crucially, it captures both the text rationale and per-token log-probabilities. 2. extractor.py: Real-world LLM outputs are notoriously messy. We engineered a 6-stage priority cascade that handles LaTeX boxes, bold markdown, currency symbols, and verbal summaries. It passes 11 out of 11 unit tests with 100% extraction recovery. 3. ca_asc_evaluator.py: This is our high-throughput offline replay simulator. By caching all full generation traces, intermediate tokens, and logprobs locally, we can simulate different stopping thresholds, margins, and batch sizes in 0.05 seconds without spending API credits or making duplicate network calls."

Key Emphasis: Mention that offline replay enables running zero-cost parameter sweeps in 0.05 seconds.

Slide 9: Experimental Setup & Baselines

[Slide Cue]: *Experimental parameters table on left; Evaluated comparison baselines on right.*

"Slide 9 outlines our rigorous experimental setup: We benchmarked on the official OpenAI GSM8K (Grade School Math 8K) test set. Our backbone model is openai/gpt-oss-20b, running on Groq LPU hardware acceleration. Sampling parameters were set to Temperature T=0.70 and Top-P p=0.95 to ensure reasoning diversity, with a 256 completion token cap per path. Our stopping policy used batch size b=2, margin lead=2, confidence threshold=70%, and a maximum budget of N=6. We evaluated our method against three standard baselines: 1. Single-Path Greedy (N=1): standard deterministic output. 2. Fixed Self-Consistency (N=1 to 6): the Wang et al. baseline. 3. Confidence-Weighted Self-Consistency (Fixed N=6): logprob-weighted majority voting. 4. Adaptive Self-Consistency (Our proposed method). Before showing benchmark results, let us address a critical parsing challenge we had to solve."

Key Emphasis: Point out that temperature T=0.70 is standard to induce diverse reasoning paths across samples.

Slide 10: Answer Extraction & Error Mitigation

[Slide Cue]: *Output formatting anomalies on left; 6-Stage Priority Cascade on right card.*

*"When benchmarking LLMs, poor parsing often leads to false negatives — the model calculates the right answer, but the evaluation script records a zero because it cannot extract the number. As shown on the left, we encountered five major formatting anomalies: 1. LaTeX boxed expressions: \boxed{1,250.00} 2. Markdown bolding with currency: **$104** 3. Verbal conclusions: 'Total: 104 dollars' 4. Trailing punctuation noise: 'The answer is 18.' 5. Unformatted rationales ending with calculations. To fix this, we implemented the 6-stage priority cascade shown on the right: Priority 1 checks for official GSM8K tags (####). Priority 2 strips LaTeX \boxed{} commands. Priority 3 matches verbal conclusion phrases ('the answer is'). Priority 4 strips markdown bold syntax. Priority 5 extracts written summary numbers. Priority 6 safely falls back to the last valid numerical token. This parser achieved 100% extraction recovery across all 180 paths in our benchmark, eliminating any risk of parsing penalties. I will now hand over to Abhinav Kumar to present our empirical results, Pareto curves, and project roadmap."*

Key Emphasis: Verbal handoff: 'I will now hand over to Abhinav Kumar to present our empirical results, Pareto curves, and project roadmap.'

PART 3: RESULTS, PARETO FRONTIER & ROADMAP | Speaker: Abhinav Kumar (Slides 11 – 16)

Slide 11: Replication of Wang et al. vs. Adaptive Self-Consistency (Figure 1)

[Slide Cue]: *Figure 1 embedded chart on left; Analytical insights breakdown on right.*

"Thank you, Aagman. Good morning respected faculty members. Slide 11 presents our first major empirical result: replicating the Wang et al. scaling curves on GSM8K and comparing them directly against our Adaptive Self-Consistency. Look closely at the blue curve on Figure 1: At N=1 (single path), accuracy is 50.0%. At N=3, accuracy jumps to 60.0% as majority voting filters out early calculation errors. At N=4 and N=5, accuracy peaks at 63.3% — an absolute gain of +13.3% over greedy decoding. However, look at N=6: accuracy actually drops back to 60.0%. This confirms our earlier hypothesis: excessive sampling at T=0.7 introduces noisy rationales that dilute consensus. Now, look at the red star: that is our Adaptive Self-Consistency. It achieves the exact same 60.0% benchmark accuracy, but it does so using an average of only 3.1 paths per question instead of 6! Because our method stops when consensus is reached, it avoids the noisy tail entirely. Let us look at what this means for token compute."

Key Emphasis: Direct faculty's attention to the red star on Figure 1: 60.0% accuracy reached at an average of k=3.1 paths.

Slide 12: Compute Efficiency: 48.4% Token Reduction on the Pareto Frontier (Figure 2)

[Slide Cue]: *Figure 2 Pareto frontier on left; '-48.4% TOKENS SAVED' badge and breakdown on right.*

"Slide 12 demonstrates our core efficiency breakthrough: The Inference Compute Pareto Frontier. In Figure 2, the horizontal axis measures the average completion tokens consumed per question, while the vertical axis shows benchmark accuracy. An ideal algorithm lies as far up and to the left as possible. Notice the comparison: Fixed Self-Consistency (N=6) consumes 1,123 tokens per problem to reach 60.0% accuracy. Our Adaptive Self-Consistency consumes only 601 tokens per problem for the exact same 60.0% accuracy. This represents a net reduction of 522 completion tokens per question — an empirical savings of 48.4%. As you can see, our data point strictly dominates the fixed curve. We achieve identical reasoning capability at almost half the computational budget, which directly translates into cutting API operational expenses by half and halving user latency."

Key Emphasis: Highlight Pareto dominance: point to the red data point lying strictly to the left of the fixed curve.

Slide 13: Sample Allocation: How the System Adapts to Problem Difficulty (Figure 3)

[Slide Cue]: *Figure 3 sample allocation histogram on left; Tiered difficulty breakdown (Easy: 66.7%, Med: 13.3%, Hard: 20.0%) on right.*

"A natural question the committee might ask is: 'Where do these savings actually come from?' Figure 3 answers this by showing the distribution of sample budgets across our test set: Easy Tier (k = 2 paths): 20 out of 30 questions (66.7%) terminated immediately on the very first batch! For two-thirds of all problems, the first two paths agreed with high confidence, saving 66.7% of compute on those questions. Medium Tier (k = 4 paths): 4 questions (13.3%) split 1-vs-1 on the first batch. The second batch broke the tie, reached consensus, and stopped safely. Complex Tier (k = 6 paths): 6 questions (20.0%) were difficult multi-constraint problems that genuinely required the full budget. The key takeaway is that our system dynamically allocates compute according to question difficulty: easy questions get 2 paths, hard questions get 6 paths, leading to our overall average of 3.1 paths per question."

Key Emphasis: Emphasize that 66.7% of questions resolve on batch 1; this is the primary driver of the 48.4% token compute reduction.

Slide 14: Ablation Study: Validating the Stopping Thresholds

[Slide Cue]: *Ablation sweep grid on left; Policy trade-off analysis on right.*

"To rigorously validate our parameter choices, we performed an extensive ablation sweep using our offline replay engine across different batch sizes, margins, and confidence levels. The results in the table highlight the critical trade-offs: If we set Margin = 1 (Row 1): the policy stops after just 1 path of agreement. While this saves 56.1% compute, it suffers a 10.0% regret rate — meaning it prematurely locks in lucky early errors that full sampling would have corrected. If we set Margin = 3 (Row 5): the policy becomes overly cautious. It waits until an average of 5.5 paths, recovering only 5.2% token savings. Now look at Row 4 (Our Selected Setting: Batch = 2, Margin = 2, Confidence = 70%): It achieves 70.0% accuracy with 0.0% regret rate and 23.0% token compute savings. This confirms that a lead margin of 2 combined with a 70% confidence share is the optimal mathematical boundary for zero-regret early stopping."

Key Emphasis: Show Row 4 in the table. Contrast the 10.0% regret rate of Margin=1 with the 0.0% regret rate of Margin=2.

Slide 15: Mid-Term Milestones Completed & Final-Term Roadmap

[Slide Cue]: *Completed milestones on left card; 4-point Final-Term roadmap on right card.*

"On Slide 15, we summarize our project milestones and next steps: As of this mid-term evaluation, we are 100% on schedule: We replicated Wang et al.'s scaling dynamics on GSM8K; built a production-grade async inference harness with exponential backoff and logprob extraction; engineered a 6-stage regex parser achieving 100% extraction recovery; empirically proved a 48.4% token compute reduction with 0.0% accuracy regret; and developed an offline replay simulator running policy sweeps in 0.05 seconds. For our Final-Term roadmap leading up to May 2027, we plan to: 1. Expand evaluation across SVAMP, MATH, and AQuA-RAT to verify cross-domain generalization. 2. Implement Token-Level Early Stopping, pruning faulty reasoning paths mid-generation before they finish. 3. Introduce Model Cascade Routing, routing easy problems (k=2) to an 8B model, reserving 70B models strictly for unresolved hard problems. 4. Submit our finalized findings to a peer-reviewed NLP / AI conference workshop (such as ACL, EMNLP, or NeurIPS)."

Key Emphasis: Show strong academic ambition: mention SVAMP/MATH evaluation, token-level pruning, and workshop submission.

Slide 16: Key Takeaways & Faculty Discussion

[Slide Cue]: *3 KPI cards (-48.4%, 66.7%, 0.0%) at top; Supervisor & Team credits card at bottom.*

"To conclude our mid-term presentation, here are the three core takeaways: 1. 48.4% Compute Reduction: We cut token usage from 1,128 tokens down to 601 tokens per question for identical benchmark accuracy. 2. 66.7% Instant Resolution: Two-thirds of reasoning problems terminate on the very first pair of samples, eliminating redundant compute. 3. 0.0% Accuracy Regret: Our dual margin-confidence rule guarantees that early stopping never sacrifices correctness. We extend our sincere gratitude to our supervisor, Dr. Bharat Singh, for his continuous guidance throughout this work, and to the Department of CSE at IIIT Ranchi. We are now open for questions, critique, and faculty discussion. Thank you!"

Key Emphasis: Polite and professional closing: thank Dr. Bharat Singh and invite faculty discussion.

FACULTY Q&A; DEFENSE PREPARATION | Anticipated Questions & Assigned Responders

Q1: What if the model is confident but produces an incorrect answer?

Responder: Aagman (Speaker 2)

That is precisely why we implemented dual stopping gates rather than relying on confidence alone. In greedy decoding, an LLM often assigns high probability to an arithmetic mistake. However, in self-consistency, different paths make different errors. For an early exit to occur, the exact same incorrect number must be generated independently by two separate paths (margin >= 2) AND must account for >= 70% of total confidence. In our benchmark, this combination prevented 100% of premature errors, achieving 0.0% regret.

Q2: Why did you choose a batch size of 2 instead of sampling 1 path at a time?

Responder: Aagman (Speaker 2) or Pankaj (Speaker 1)

A batch size of 2 is the theoretical minimum required to evaluate a lead margin of 2. If we sampled 1 path at a time (sequential b=1), each step would introduce network latency overhead for a single API call. Sampling in pairs of 2 utilizes concurrent requests on Groq hardware, maximizing GPU throughput while allowing the earliest possible consensus exit at k=2.

Q3: How does your method compare to test-time compute models like OpenAI o1 / o3?

Responder: Abhinav Kumar (Speaker 3)

Models like o1 use reinforcement learning (RL) to generate very long internal reasoning traces (often thousands of tokens). Our method is an orthogonal, black-box inference strategy that works on any standard instruction-tuned model without RL retraining. Furthermore, adaptive stopping principles can directly complement models like o1 by dynamically allocating thinking budget instead of spending maximum tokens unconditionally.

Q4: Have you tested this on harder benchmarks like the MATH dataset?

Responder: Abhinav Kumar (Speaker 3)

For the mid-term phase, we focused on establishing our empirical baseline and verifying the 48.4% reduction on GSM8K. As outlined in our final-term roadmap on Slide 15, evaluating across MATH, SVAMP, and AQuA-RAT is our immediate priority for the final phase to evaluate how difficulty distributions shift the average budget.

Q5: How do you extract token log-probabilities from modern LLM APIs?

Responder: Aagman (Speaker 2)

We enable the logprobs parameter in the OpenAI-compatible API call (setting logprobs=True and top_logprobs=1). For each completion token, the API returns the logarithm of the probability assigned by the model. We sum these log-probabilities across the generated sequence and divide by the token count to obtain length-normalized sequence log-probability.

PRESENTATION DELIVERY TIPS | Best Practices for Faculty Defense

• **Slide Transitions:** Practice the two handoffs out loud. Pankaj hands off to Aagman at Slide 5; Aagman hands off to Abhinav at Slide 10. Avoid dead air or awkward pauses.

• **Pointing at Figures:** When Abhinav presents Slides 11 and 12, point directly to the red star (Adaptive SC) vs. the blue curve (Wang et al.), emphasizing the 522 token delta.

• **Pacing & Eye Contact:** Spend approximately 45 to 60 seconds per slide. Make eye contact with Dr. Bharat Singh and panel members; avoid reading directly from the screen.

• **Confidence on Numbers:** Memorize the three key stats: **-48.4% compute reduction**, **66.7% batch-1 resolution**, and **0.0% accuracy regret**.